Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-207БВ-25

Студент: Михеева А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 16.10.26

Москва, 2026

Постановка задачи

Вариант 20.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

Дан массив координат (x, y, z). Необходимо найти три точки, которые образуют треугольник максимальной площади.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- pthread_t, pthread_create(pthread_t *thread, const pthread_attr_t *attr, void (start_routine)(void *), void *arg); — создаёт новый поток, который начинает выполнение с функции start_routine, принимающей аргумент arg.
- int pthread_join(pthread_t thread, void **retval); — блокирует вызывающий поток до завершения указанного потока thread. Возвращает результат выполнения потока через retval.
- pthread_mutex_t, PTHREAD_MUTEX_INITIALIZER — статическая инициализация мьютекса.
- int pthread_mutex_lock(pthread_mutex_t *mutex); — захватывает мьютекс. Если мьютекс уже захвачен другим потоком, вызывающий поток блокируется.
- int pthread_mutex_unlock(pthread_mutex_t *mutex); — освобождает мьютекс.
- FILE *fopen(const char *pathname, const char *mode); — открывает файл.
- int fscanf(FILE *stream, const char *format, ...); — читает форматированные данные из файла.
- int fclose(FILE *stream); — закрывает файл.
- double sqrt(double x); — вычисляет квадратный корень.
- int clock_gettime(clockid_t clk_id, struct timespec *tp); — возвращает текущее значение времени заданных часов. Используется CLOCK_MONOTONIC — монотонные часы, не подверженные изменениям системного времени.
- void *malloc(size_t size); — выделяет память в куче.
- void free(void *ptr); — освобождает память.

Площадь треугольника в трёхмерном пространстве вычисляется через векторное произведение: для точек $A(x_1, y_1, z_1)$, $B(x_2, y_2, z_2)$, $C(x_3, y_3, z_3)$ строятся векторы AB и AC. Площадь треугольника равна половине модуля вектора векторного произведения AB * AC.

Программа состоит из двух отдельных исполняемых файлов: sequential и parallel.

sequential — последовательная версия. Открывает файл с точками, читает количество точек n_points и сами координаты, затем тройным циклом перебирает все тройки (i, j, k), где $0 \le i < j < k <$ n_points. Для каждой тройки вычисляется площадь, и если она больше текущего максимума, максимум обновляется. Время работы замеряется через clock_gettime до и после цикла. Результат выводится в стандартный поток вывода.

parallel — параллельная версия. Открывает файл, читает точки. Создаёт n_threads потоков через pthread_create. Каждый поток получает структуру ThreadArgs, содержащую указатель на массив точек, количество точек N и диапазон индексов start и end первого индекса i. Разбиение диапазона i = 0..N - 3 между n_threads потоками выполняется без пропусков и пересечений по формуле n_points - 2) / (int)n_threads + ((int)i < (n_points - 2) % (int)n_threads ? 1 : 0). Каждый поток обрабатывает только свой диапазон i, а внутренние индексы j и k пробегает сам. Локальный максимум потока сохраняется в локальных переменных, а затем под мьютексом сравнивается с глобальным максимумом global_best. Мьютекс защищает только критическую секцию обновления глобального максимума, а не весь перебор, что позволяет потокам работать параллельно большую часть времени. После завершения всех потоков главный поток вызывает pthread_join для каждого потока, вычисляет время работы и выводит результат.

Массив точек только читается потоками, запись в него происходит до создания потоков в главном потоке, поэтому мьютекс для защиты массива не нужен.

Для сравнения последовательной и параллельной версий используются время выполнения, ускорение и эффективность. Время последовательной версии в проведённом эксперименте составило T_s = 5833,360 мс.

Ускорение показывает, во сколько раз параллельная программа выполняется быстрее последовательной. Оно вычисляется по формуле

$$S = \frac{T_s}{T_p},$$

где S — ускорение; T_s — время выполнения последовательной программы; T_p — время выполнения параллельной программы при заданном количестве потоков. Если S больше 1, параллельная версия быстрее последовательной; если S меньше 1, параллельная версия работает медленнее.

Эффективность показывает, насколько результативно используются потоки при параллельном выполнении. Она вычисляется по формуле:

$$X = \frac{S}{p}$$

где X — эффективность; S — ускорение; p — количество потоков. Значение эффективности, близкое к 1, соответствует близкому к идеальному ускорению для заданного числа потоков. При росте числа потоков эффективность может снижаться из-за накладных расходов и ограниченности аппаратных ресурсов.

В таблице приведены измеренные значения времени выполнения, ускорения и эффективности для различных количеств потоков. В качестве базового времени для расчёта ускорения использовано время последовательной программы T_s = 5833,360 мс.

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	5922,983	0,985	0,985
2	5363,840	1,088	0,544

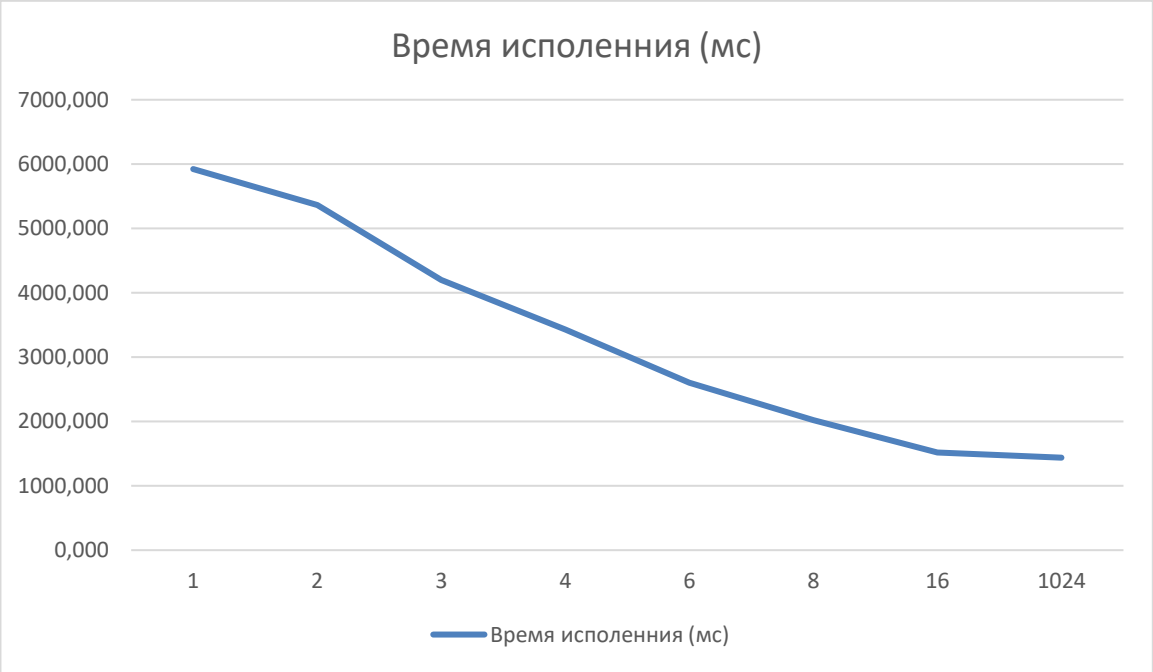
3	4199,839	1,389	0,463
4	3430,392	1,700	0,425
6	2604,700	2,240	0,373
8	2019,740	2,888	0,361
16	1516,656	3,846	0,240
1024	1437,196	4,059	0,004

По результатам измерений время выполнения в целом уменьшается при увеличении количества потоков. При одном потоке параллельная версия выполняется за 5922,983 мс, что немного медленнее последовательной версии (5833,360 мс). Это объясняется накладными расходами на создание и организацию потока.

При переходе от 16 к 1024 потокам время уменьшается лишь до 1437,196 мс. Это показывает, что при очень большом количестве потоков дополнительное ускорение становится небольшим.

Ускорение увеличивается от 1,088 при двух потоках до 4,059 при 1024 потоках. Максимальное измеренное ускорение составляет около 4,06 раза. При этом рост не пропорционален количеству потоков: например, увеличение их числа с 16 до 1024 повышает ускорение лишь с 3,846 до 4,059.

Эффективность уменьшается с ростом количества потоков, следовательно, дополнительные потоки дают всё меньший прирост производительности на каждый поток. Причиной является то, что количество потоков может превышать число доступных аппаратных вычислительных ресурсов. В этом случае потоки конкурируют за процессорное время, а расходы на планирование и переключение снижают полезный эффект параллелизации. Ускорение при этом ещё может немного расти, но эффективность падает.







Код программы

```
twentyeighth@connor:~/lab$ ls
parallel  parallel.c  points.txt  sequential  sequential.c
twentyeighth@connor:~/lab$ cat parallel.c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <pthread.h>
```

```
	typedef struct {
		double x, y, z;
	} coords;

	typedef struct {
		coords* point;
		int n;
		int start, end;
	} ThreadArgs;

	static double global_best = 0.0;
	static int gbi = 0, gbj = 0, gbl = 0;
	static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

	static double triangle(coords a, coords b, coords c) {
		coords ab = { b.x – a.x, b.y – a.y, b.z – a.z };
		coords ac = { c.x – a.x, c.y – a.y, c.z – a.z };
		coords ab_ac = { ab.y * ac.z – ab.z * ac.y, ab.z * ac.x – ab.x * ac.z, ab.x *
ac.y – ab.y * ac.x };
		return 0.5 * sqrt(ab_ac.x * ab_ac.x + ab_ac.y * ab_ac.y + ab_ac.z * ab_ac.z);
	}

	static double get_time_ms(void) {
		struct timespec ts;
		clock_gettime(CLOCK_MONOTONIC, &ts);
		return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
	}

	static void* work(void* _args) {
		ThreadArgs* args = (ThreadArgs*)_args;

		double local_best = 0.0;
		int lbi = 0, lbj = 0, lbl = 0;

		for (int i = args–>start; i < args–>end; i++)
```

```
                for (int j = i + 1; j < args->n - 1; j++)
                        for (int l = j + 1; l < args->n; l++) {
                                double s = triangle(args->point[i], args->point[j], args->point[l]);
                                if (s > local_best) {
                                        local_best = s;
                                        lbi = i;
                                        lbj = j;
                                        lbl = l;
                                }
                        }

        pthread_mutex_lock(&mutex);
        if (local_best > global_best) {
                global_best = local_best;
                gbi = lbi;
                gbj = lbj;
                gbl = lbl;
        }
        pthread_mutex_unlock(&mutex);

        return NULL;
}

int main(int argc, char** argv) {
        if (argc != 3) {
                fprintf(stderr, "usage: %s points.txt thread_count\n", argv[0]);
                return 1;
        }

        size_t n_threads = atoi(argv[2]);
        if (n_threads < 1 || n_threads > 4096) {
                fprintf(stderr, "error: invalid thread count\n");
                return 1;
        }
```

```
	FILE* file = fopen(argv[1], "r");
	if (!file) {
		fprintf(stderr, "fopen failed\n");
		return 1;
	}

	int n_points;
	if (fscanf(file, "%d", &n_points) != 1 || n_points < 3) {
		fprintf(stderr, "error: invalid number of points\n");
		fclose(file);
		return 1;
	}

	coords* points = malloc(n_points * sizeof(coords));
	if (!points) {
		fclose(file);
		return 1;
	}

	for (int i = 0; i < n_points; ++i) {
		if (fscanf(file, "%lf %lf %lf", &points[i].x, &points[i].y, &points[i].z)
!= 3) {
			fprintf(stderr, "error: invalid point #%d\n", i);
			free(points);
			fclose(file);
			return 1;
		}
	}
	fclose(file);

	pthread_t* threads = malloc(n_threads * sizeof(pthread_t));
	ThreadArgs* thread_args = malloc(n_threads * sizeof(ThreadArgs));

	int start = 0;
	double t0 = get_time_ms();
```

```
	for (size_t i = 0; i < n_threads; i++) {
		int chunk = (n_points - 2) / (int)n_threads
			+ ((int)i < (n_points - 2) % (int)n_threads ? 1 : 0);
		thread_args[i] = (ThreadArgs){
			.point = points,
			.n = n_points,
			.start = start,
			.end = start + chunk
		};
		start += chunk;
		pthread_create(&threads[i], NULL, work, &thread_args[i]);
	}

	for (size_t i = 0; i < n_threads; i++)
		pthread_join(threads[i], NULL);

	double t = get_time_ms() - t0;

	printf("S:\t%.6f\n", global_best);
	printf("points:\t%d %d %d\n", gbi, gbj, gbl);
	printf("time:\t%.3f\n", t);

	free(thread_args);
	free(threads);
	free(points);
	return 0;
}
twentyeighth@connor:~/lab$ cat sequential.c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

typedef struct {
```

```
        double x, y, z;
    } coords;

    static double triangle(coords a, coords b, coords c) {
        coords ab = { b.x − a.x, b.y − a.y, b.z − a.z };
        coords ac = { c.x − a.x, c.y − a.y, c.z − a.z };
        coords ab_ac = { ab.y * ac.z − ab.z * ac.y, ab.z * ac.x − ab.x * ac.z, ab.x *
ac.y − ab.y * ac.x };
        return 0.5 * sqrt(ab_ac.x * ab_ac.x + ab_ac.y * ab_ac.y + ab_ac.z * ab_ac.z);
    }

    static double get_time_ms(void) {
        struct timespec ts;
        clock_gettime(CLOCK_MONOTONIC, &ts);
        return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
    }

    int main(int argc, char** argv) {
        if (argc != 2) {
            fprintf(stderr, "usage: %s points.txt\n", argv[0]);
            return 1;
        }

        FILE* file = fopen(argv[1], "r");
        if (!file) {
            fprintf(stderr, "fopen failed\n");
            return 1;
        }

        int n_points;
        if (fscanf(file, "%d", &n_points) != 1 || n_points < 3) {
            fprintf(stderr, "error: invalid number of points\n");
            fclose(file);
            return 1;
        }
```

```
	coords* points = malloc(n_points * sizeof(coords));
	if (!points) {
		fclose(file);
		return 1;
	}

	for (int i = 0; i < n_points; ++i) {
		if (fscanf(file, "%lf %lf %lf", &points[i].x, &points[i].y, &points[i].z)
!= 3) {
			fprintf(stderr, "error: invalid point #%d\n", i);
			free(points);
			fclose(file);
			return 1;
		}
	}
	fclose(file);

	double t0 = get_time_ms();

	double maxS = 0.0;
	int bi = 0, bj = 0, bl = 0;
	for (int i = 0; i < n_points - 2; ++i)
		for (int j = i + 1; j < n_points - 1; ++j)
			for (int l = j + 1; l < n_points; ++l) {
				double s = triangle(points[i], points[j], points[l]);
				if (s > maxS) {
					maxS = s;
					bi = i;
					bj = j;
					bl = l;
				}
			}

	double t = get_time_ms() - t0;

	printf("S:\t%.6f\n", maxS);
```

```
	printf("points:\t%d %d %d\n", bi, bj, bl);
	printf("time:\t%.3f\n", t);


	free(points);
	return 0;
}
```

Протокол работы программы

```
twentyeighth@connor:~/lab$ ./sequential points.txt
S:	748742.760986
points:	66 1748 1905
time:	5833.360
twentyeighth@connor:~/lab$ ./parallel points.txt 1
S:	748742.760986
points:	66 1748 1905
time:	5922.983
twentyeighth@connor:~/lab$ ./parallel points.txt 2
S:	748742.760986
points:	66 1748 1905
time:	5363.840
twentyeighth@connor:~/lab$ ./parallel points.txt 3
S:	748742.760986
points:	66 1748 1905
time:	4199.839
twentyeighth@connor:~/lab$ ./parallel points.txt 4
S:	748742.760986
points:	66 1748 1905
time:	3430.392
twentyeighth@connor:~/lab$ ./parallel points.txt 6
S:	748742.760986
points:	66 1748 1905
time:	2604.700
twentyeighth@connor:~/lab$ ./parallel points.txt 8
S:	748742.760986
points:	66 1748 1905
time:	2019.740
```

```
twentyeighth@connor:~/lab$ ./parallel points.txt 16
S:       748742.760986
points:  66 1748 1905
time:    1516.656
twentyeighth@connor:~/lab$ ./parallel points.txt 1024
S:       748742.760986
points:  66 1748 1905
time:    1437.196
```

Вывод

В ходе лабораторной работы была разработана и исследована программа на языке C для поиска трёх точек в трёхмерном пространстве, образующих треугольник максимальной площади. Были рассмотрены последовательный и параллельный варианты выполнения задачи, а также измерено время работы параллельной программы при различном количестве потоков.

Результаты показали, что увеличение числа потоков позволяет уменьшить время вычислений. Минимальное время среди представленных измерений составило 1437,196 мс при 1024 потоках, а максимальное ускорение — 4,059 раза. Однако дополнительный выигрыш при переходе от 16 к 1024 потокам оказался небольшим. Эффективность снизилась с 0,544 при двух потоках до 0,004 при 1024 потоках. Чрезмерное увеличение количества потоков не гарантирует пропорционального ускорения из-за накладных расходов и ограничений аппаратных ресурсов. Полученные результаты демонстрируют преимущества многопоточности, а также ограничения, возникающие при использовании чрезмерного количества потоков.